

Advanced Touying

The Touying Authors

01.01.2042

Abstract

This document is the article output of the example `advanced.typ` and showcases all the normal presentation does as well but written in prose. See at the [end](#) or the source directly for how that is done.

Animating mathematics

Since touying `v0.8.0` math is now fully supported.

Marks reach inside math elements

Animation marks reach into the math elements that store their content in named fields — `frac`, `mat`, `vec`, `cases`, `binom`, `root`, `attach`, `accent`, `cancel`, `lr` and the brace family — so the numerator is revealed in place. In this article every stage is already shown.

Written as:

```
$ frac(a #pause + b, c) $
```

or, cell by cell:

```
$ mat(a, #pause b; c, #pause d) $
```

More in [Docs/Math Equation Animations](#).

Building a derivation

`#uncover` and `#only` work inside math as well as around it. `#uncover` keeps the space a term will occupy so the equation does not reflow as it is revealed, while `#only` removes the space entirely — which is why a line gated with it arrives without having reserved room first. The finished derivation is $f(x) = x^2 + 2x + 1 = (x + 1)^2$.

Written as:

```
$ f(x) &= x^2 #uncover("2-")[$+ 2x$] #uncover("3-")[$+ 1$] \
    #only("3-")[$&= (x + 1)^2$] $
```

Choosing between versions

`alternatives` and its cousins

`#alternatives` shows one body per subslide and reserves the space of the largest with `stretch: true`, so nothing around it moves; `#alternatives-cases` drives the same mechanism from a function, which e.g. suits highlighting one row of a list at a time. An article has no subslides, so both collapse to their final state.

That may look like this:

```
#alternatives(stretch: true)[first][second][third]
```

or this

```
#alternatives-cases(("3", "4", "5"), case => {  
  let rows = ("chlorophyll", "carotene", "anthocyanin")  
  stack(dir: ltr, spacing: 1.2em,  
    ..rows.enumerate().map((i, r)) => {  
      if i == case { strong(r) } else { text(fill: gray, r) }  
    }),  
  })  
})
```

More in [Docs/Complex Animations](#).

animate composes effects

#animate layers effects on one body, each with its own subslide range and priority: placements choose whether and what body appears at all, while styling functions wrap whatever the placement left. Multiple styling effects may stack on top of each other.

There are four placements — **"show"**, **"cover"**, **"remove"** and **swap(..)**. These are modeled after uncover, only and alternatives. The default effect is **"show"** over subslides **"1-"** at priority **0**, which is why an unmarked body is visible from the first subslide. The last effect written wins a tie, and **priority** overrides that ordering.

That may look like this:

```
#animate(  
  [Photosynthesis],  
  effects: (  
    (effect: swap([Respiration], stretch: true), subslides: "2-"),  
    (effect: (body, self: none) => text(fill: red, body), subslides: "3-"),  
    (effect: swap([Photosynthesis], stretch: true), subslides: 4, priority: 2)  
  ),  
)
```

Reusing content

Waypoints name positions

A waypoint labels a position in a slide's flow so that animations refer to a name rather than a number, which keeps them correct when a **#pause** is inserted earlier. Markers such as **get-first**, **get-last**, **from-wp** and **until-wp** derive ranges from it. A waypoint captures every subslide up to the next one, and **advance: false** marks the current position instead of claiming a new subslide of its own.

That may look like this:

```
#waypoint(<detail>, advance:false)  
#uncover(<detail>)[This line names `<detail>`, not "subslide 2".]  
#pause  
#only(get-last(<detail>))[`get-last` pins the waypoint's final subslide – useful  
when a range should end exactly where a phase does.]  
#waypoint(<other-stuff>)  
Waypoints capture all subslides up until the next waypoint.
```

More in [Docs/Waypoints](#).

Recalling a slide

Labelling a slide and calling `#touying-recall(<label>)` replays it later in the presentation. Because a label belongs to the slide rather than to its heading, several slides under one heading remain individually addressable. In an article there are no slides to replay, so a recall of a whole slide is skipped.

That may look like this:

```
== Shared heading
#slide[.. first ..]<intro-a>
#slide[.. second ..]<intro-b>

#touying-recall(<intro-b>)
```

Rendering Content

`#touying-render` takes content held in a variable and renders one stage of it, so an animated element can be shown at a chosen point without being written out again. `subslides` picks the stage: an index, a range, a negative index counting back from the last, or a waypoint. The content need not be a single element — a whole reveal sequence works, and individual steps can be left out.

That may look like this:

```
#let my-table = table(
  columns: 2,
  [Header 1], [Header 2],
  pause, [Cell 1],
  [Cell 2], pause,
  [Cell 3], [Cell 4],
)

#touying-render(my-table, subslides: 2)
```

More in [Docs/Complex Animations](#).

Layout and graphics

Animating a drawing

`#touying-reduce` connects a drawing package such as `cetz` or `fletcher` to Touying's animation system, so a `pause` between draw commands splits the canvas across subslides while keeping one coordinate space. The article shows the completed drawing. Touying supplies the package's own cover function, so a hidden part keeps its size and the drawing does not shift as it is revealed. Bindings ship for `cetz`, `fletcher` and `alchemist`; other packages have to expose their own. `#touying-diagram` is a synonym, named for the diagrams these packages usually draw.

That may look like this:

```
#let cetz-canvas = touying-reduce.with(cetz)

#cetz-canvas({
  import cetz.draw: *
  circle((0, 0), radius: 1)
```

```
(pause,)
line((-1, 0), (1, 0))
})
```

More in [Docs/CeTZ integration](#).

Two columns, and what the article does with them

A slide can be passed multiple bodies which will be layed out in columns. Article mode linearizes them into running text, since a flowing document has no slide to divide. Two bodies split a slide in half by default; an explicit `composer` such as `(2fr, 1fr)` divides it differently.

Wrap a container in `#article-keep-layout[...]` to keep its shape in the article, or `#article-linearize[...]` to force the opposite; both work on columns, grids and tables. Tables and grids are linearized by default unless they carry a header or footer.

That may look like this:

```
#slide(composer: (2fr, 1fr))[left][right]
```

More in [Docs/Article Mode](#).

One source, multiple outputs

Selecting what goes where.

Five markers decide where content goes. `#presentation-only[...]`, `#handout-only[...]` and `#article-only[...]` keep content to one output; `#slides-only[...]` routes it to both the presentation and handout. `#article-text[...]` replaces a slide's body with prose written for reading, which is what nearly every slide in this deck does. The same filtering of `...only` is available as `<touying:...>` labels on a heading or a slide see [Output Modes](#).

You are reading the article, so this paragraph, written inside `#article-only[...]`, is the one that survives.

The rule of thumb: write the slide for projection, then add `#article-text[...]` wherever the prose version needs to differ. It replaces the entire slide content with written prose. See the docs or this source.

Handout mode

A handout is a slide output with the animations flattened: each slide keeps by default only its final subslide, so this deck's presentation pages collapse to roughly a third. `#presentation-only[...]` and `#handout-only[...]` divide content between the live talk and the printed version. Compile a handout with `--input export-mode=handout`, or set `handout: true` in the global `config-common`.

`config-common(handout-subslides: ...)` keeps particular subslides rather than only the last; set it with `#touying-set-config` or pass it as an argument to a `#slide` call.

That may look like this:

```
#presentation-only[.. only in the live talk ..]  
#handout-only[.. only in the printed handout ..]  
  
#slide(config: config-common(handout-subslides: (1, 3)))[.. ..]
```

More in [Docs/Handout Mode](#).